PHASE 14 OF 18

Collections II: Maps & Concurrent Collections

The Map API, HashMap internals (hashing, buckets, collisions, treeification, resizing), custom keys, LinkedHashMap and an LRU cache, TreeMap navigation, Hashtable, IdentityHashMap, WeakHashMap, ConcurrentHashMap, sequenced collections, choosing a collection, and Big-O cheat sheets

IN THIS PHASE

14.1 The Map Interface
14.2 HashMap Internals
14.3 Custom Objects as Keys
14.4 LinkedHashMap: Predictable Order and an LRU Cache
14.5 TreeMap: Sorted Keys and Range Queries
14.6 Other Map Implementations
14.7 Map Implementations Compared
14.8 Thread-Safe Collections
14.9 Sequenced Collections (Java 21)
14.10 Choosing the Right Collection
14.11 Big-O Cheat Sheet
14.12 Classic Map Programs
14.13 Predict the Output
14.14 Interview Questions
14.15 Quick Revision

14.1 The Map Interface

A **Map** stores **key → value** pairs (entries). Keys are **unique**; values may repeat. `Map` is part of the Collections Framework but does **not** extend `Collection`.

```
Map<String, Integer> stock = new HashMap<>();
stock.put("APPLE", 50);                 // add
stock.put("MANGO", 20);
stock.put("APPLE", 70);                 // same key -> value REPLACED (returns old value 50)
stock.get("APPLE");                     // 70
stock.get("KIWI");                      // null (absent)
stock.getOrDefault("KIWI", 0);          // 0
stock.containsKey("MANGO");             // true
stock.containsValue(20);                // true (O(n) scan)
stock.remove("MANGO");                  // returns 20
stock.size();                           // 1
```

Map methods (including the Java 8 additions)

Method	Purpose
`put(k, v)`	Insert or replace; returns the previous value or `null`
`get(k)`, `getOrDefault(k, def)`	Lookup
`containsKey(k)`, `containsValue(v)`	Checks (key O(1) in a HashMap; value O(n))
`remove(k)`, `remove(k, v)`	Delete (the second only if mapped to v)
`putIfAbsent(k, v)`	Insert only if the key is missing or null
`computeIfAbsent(k, k -> newV)`	Create the value lazily if missing (great for multimaps)
`computeIfPresent(k, (k, v) -> ...)`	Update only if present (returning null removes the entry)
`compute(k, (k, v) -> ...)`	Always recompute
`merge(k, v, (old, v) -> ...)`	Insert v, or combine with the existing value (counters, sums)
`replace(k, v)`, `replaceAll((k, v) -> ...)`	Update values
`keySet()`, `values()`, `entrySet()`	**Views** of keys, values and entries (removing from a view removes from the map)
`forEach((k, v) -> ...)`	Iterate
`Map.of(...)`, `Map.entry(k, v)`, `Map.copyOf(m)`, `Map.ofEntries(...)`	Immutable maps (Java 9/10); no null keys or values

```
Map<String, Integer> wordCount = new HashMap<>();
for (String w : "to be or not to be".split(" ")) {
    wordCount.merge(w, 1, Integer::sum);              // counting in one line
}
// {not=1, or=1, be=2, to=2}  (HashMap order is not guaranteed)

Map<String, List<String>> branchCustomers = new HashMap<>();
branchCustomers.computeIfAbsent("PUNE", k -> new ArrayList<>()).add("Rohit");   // multimap
branchCustomers.computeIfAbsent("PUNE", k -> new ArrayList<>()).add("Amit");
// {PUNE=[Rohit, Amit]}
```

Iterating a Map

```
Map<String, Double> balances = new LinkedHashMap<>();
balances.put("SB-1", 2500.0); balances.put("SB-2", 9100.0);

for (Map.Entry<String, Double> e : balances.entrySet()) {     // BEST: key + value together
    System.out.println(e.getKey() + " -> " + e.getValue());
}
balances.forEach((acct, bal) -> System.out.println(acct + " = " + bal));   // Java 8
for (String key : balances.keySet()) { balances.get(key); }  // works, extra lookup
balances.entrySet().removeIf(e -> e.getValue() < 5000);       // safe removal while iterating
balances.replaceAll((k, v) -> v * 1.04);                      // bulk update
```

`Map.Entry<K,V>` is the nested interface representing one pair: `getKey()`, `getValue()`, `setValue(v)` (writes through to the map, for mutable maps).

14.2 HashMap Internals

`HashMap` is an **array of buckets** (`Node<K,V>[] table`). Each node stores `hash`, `key`, `value` and `next`.

```
table (capacity 16)            a bucket holds a linked list (or a red-black tree)
+----+
| 0  | -> [hash|null|V] (the null key always lives in bucket 0)
| 1  |
| 2  | -> [hash|"Ravi"|101] -> [hash|"Amit"|104]     <- collision: same bucket index
| 3  |
| .. |
| 9  | -> [hash|"Zoya"|102]
| 15 |
+----+
```

put(key, value), step by step

1. **Hash the key**: `h = key.hashCode(); hash = h ^ (h >>> 16)`. This spreads the high bits into the low bits so that more of the hash affects the index.
2. **Find the bucket**: `index = (n - 1) & hash`, where `n` is the table size (always a power of 2). This is a fast equivalent of `hash % n`.
3. **Empty bucket** → place a new node there.
4. **Occupied bucket** → walk the nodes. If a node has the **same hash and** `equals()` key, **replace its value**. Otherwise append a new node (a **collision**).
5. If a bucket's list reaches **8 nodes** (`TREEIFY_THRESHOLD`) **and** the table capacity is **≥ 64**, the list becomes a **red-black tree** (O(log n) lookup). If capacity is below 64, the table resizes instead. A tree shrinks back to a list at **6** nodes.
6. Increment the size. If `size > capacity × loadFactor` (16 × 0.75 = **12**), **resize**: double the capacity and redistribute the nodes. Each node either stays at index `i` or moves to `i + oldCapacity`.

`get(key)` repeats steps 1–2, then checks nodes in that bucket using `hash` and `equals()`.

Constant	Default	Meaning
Initial capacity	**16** (allocated on the first `put`)	Number of buckets (always a power of 2)
Load factor	**0.75**	Resize when size exceeds capacity × 0.75
Threshold	12	First resize happens at the 13th entry
`TREEIFY_THRESHOLD`	8	List → tree in one bucket
`UNTREEIFY_THRESHOLD`	6	Tree → list
`MIN_TREEIFY_CAPACITY`	64	Below this, resize instead of treeifying

Operation	Average	Worst case
`put`, `get`, `remove`, `containsKey`	**O(1)**	O(log n) since Java 8 (tree bins); O(n) before Java 8
Iteration	O(capacity + size)	n/a

Key properties

- One `null` key (stored in bucket 0) and any number of `null` values.
- **No ordering guarantee**: the order can even change after a resize.
- Not synchronized. Concurrent modification can corrupt it or lose updates; use `ConcurrentHashMap`.
- Pre-size big maps to avoid repeated resizing: `new HashMap<>(expected * 4 / 3 + 1)`, or `HashMap.newHashMap(expected)` in Java 19+.

> **REMEMBER**
> A HashMap's correctness depends entirely on the key's `hashCode()` **and** `equals()`. Equal keys must have equal hash codes. A bad `hashCode` (for example `return 1;`) still works correctly but piles every entry into one bucket, which makes the map slow.

14.3 Custom Objects as Keys

```
final class AccountKey {                          // immutable key class
    private final String ifsc, accountNo;
    AccountKey(String ifsc, String accountNo) { this.ifsc = ifsc; this.accountNo = accountNo; }
    @Override public boolean equals(Object o) {
        return o instanceof AccountKey k
            && ifsc.equals(k.ifsc) && accountNo.equals(k.accountNo);
    }
    @Override public int hashCode() { return Objects.hash(ifsc, accountNo); }
}

Map<AccountKey, BigDecimal> ledger = new HashMap<>();
ledger.put(new AccountKey("HDFC0001", "1001"), new BigDecimal("500.00"));
ledger.get(new AccountKey("HDFC0001", "1001"));    // 500.00: found via equal key
// Without equals/hashCode -> null, because a new key object is a "different" key

record CardKey(String network, String last4) { }   // records generate equals/hashCode
Map<CardKey, String> owners = Map.of(new CardKey("VISA", "4417"), "Rohit");
```

PITFALL

If you **mutate** a key's fields after inserting it, its hash changes, so `get` looks in the wrong bucket and returns `null`, and the entry becomes unreachable (a memory leak). Use **immutable keys**: `String`, wrappers, enums, records, or final classes.

14.4 LinkedHashMap: Predictable Order and an LRU Cache

`LinkedHashMap` extends `HashMap` and adds a **doubly linked list** through all entries, so it keeps:

- **Insertion order** (the default), or
- **Access order** (`new LinkedHashMap<>(cap, 0.75f, true)`): each `get`/`put` moves the entry to the end.

```
Map<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("C", 3); lhm.put("A", 1); lhm.put("B", 2);
System.out.println(lhm);                  // {C=3, A=1, B=2}  insertion order

class LruCache<K, V> extends LinkedHashMap<K, V> {          // classic interview question
    private final int capacity;
    LruCache(int capacity) {
        super(16, 0.75f, true);                              // accessOrder = true
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;                            // evict the least recently used
    }
}

LruCache<String, String> cache = new LruCache<>(2);
cache.put("A", "1"); cache.put("B", "2");
cache.get("A");                           // A becomes most recently used
cache.put("C", "3");                      // evicts B (least recently used)
System.out.println(cache.keySet());       // [A, C]
```

14.5 TreeMap: Sorted Keys and Range Queries

`TreeMap` stores entries in a **red-black tree** sorted by key (natural order or a `Comparator`). Operations are **O(log n)**. Null keys aren't allowed (null values are). It implements `NavigableMap`.

```
TreeMap<Integer, String> tm = new TreeMap<>(Map.of(30, "C", 10, "A", 20, "B"));
System.out.println(tm);                    // {10=A, 20=B, 30=C}  sorted by key
tm.firstKey();  tm.lastEntry();            // 10, 30=C
tm.headMap(20); tm.tailMap(20);            // {10=A}, {20=B, 30=C}
tm.subMap(10, 30);                         // {10=A, 20=B}  [from, to)
tm.descendingMap();                        // {30=C, 20=B, 10=A}
tm.floorKey(25); tm.ceilingKey(25);        // 20, 30

// Real use: slab lookup (fees or interest by amount slab)
TreeMap<Integer, Double> fdRates = new TreeMap<>();
fdRates.put(0, 3.5);          // 0 to 99,999      -> 3.5%
fdRates.put(100_000, 6.8);    // 1 L to 9.99 L    -> 6.8%
fdRates.put(1_000_000, 7.25); // 10 L and above   -> 7.25%
double rate = fdRates.floorEntry(250_000).getValue();   // 6.8 (greatest key <= 2.5 L)

TreeMap<String, Integer> byName = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);
```

14.6 Other Map Implementations

Hashtable (legacy)

A synchronized hash table from Java 1.0. Every method is `synchronized` (one lock for the whole table), and it allows **no null key and no null value**.

Feature	HashMap	Hashtable
Since	1.2 (Collections Framework)	1.0 (legacy)
Thread-safe	No	Yes (method-level `synchronized`)
Performance	Faster	Slower (contention)
Null keys / values	One null key, many null values	**None** (NPE)
Iteration	Fail-fast `Iterator`	`Enumeration` + Iterator
Superclass	`AbstractMap`	`Dictionary` (obsolete)
Recommendation	Single-threaded use	Use `ConcurrentHashMap` instead

IdentityHashMap

Compares keys with `==` (reference identity) and `System.identityHashCode()` instead of `equals`/`hashCode`.

```
Map<String, Integer> hm = new HashMap<>();
Map<String, Integer> ihm = new IdentityHashMap<>();
String k1 = new String("Ashok"), k2 = new String("Ashok");    // equal, different objects
hm.put(k1, 101);  hm.put(k2, 102);
ihm.put(k1, 101); ihm.put(k2, 102);
System.out.println(hm.size() + " " + ihm.size());           // 1 2
```

It is used for object-graph algorithms (serialization, deep copy) where *identity* matters.

WeakHashMap

Holds keys through **weak references**. When a key has no other strong reference, the GC may collect it, and its entry **disappears** from the map. It is used for caches and metadata attached to objects with an independent lifetime.

```
Map<Object, String> meta = new WeakHashMap<>();
Object session = new Object();
meta.put(session, "user=rohit");
session = null;                  // no strong reference left
System.gc();                     // only a REQUEST: removal is likely but not guaranteed
System.out.println(meta.size()); // usually 0 after GC (not guaranteed)
```

> **NOTE**
> `EnumMap` (Phase 10) is the fastest map when keys are enum constants. `Properties` (Phase 12) is a legacy `Hashtable<Object,Object>` subclass for String configuration.

14.7 Map Implementations Compared

Map	Ordering	Null key / values	Thread-safe	get/put	Underlying structure
`HashMap`	None	1 / many	No	O(1)	Hash table + tree bins
`LinkedHashMap`	Insertion or access	1 / many	No	O(1)	Hash table + linked list
`TreeMap`	**Sorted** by key	**No** / many	No	O(log n)	Red-black tree
`Hashtable`	None	No / No	Yes (coarse lock)	O(1)	Hash table
`ConcurrentHashMap`	None	**No / No**	**Yes** (fine-grained)	O(1)	Hash table + CAS + per-bin locks
`ConcurrentSkipListMap`	**Sorted**	No / No	Yes	O(log n)	Skip list
`EnumMap`	Enum declaration order	No / many	No	O(1)	Array indexed by ordinal
`IdentityHashMap`	None	1 / many	No	O(1)	Linear-probe table, `==` keys
`WeakHashMap`	None	1 / many	No	O(1)	Hash table with weak keys

14.8 Thread-Safe Collections

Option	How it's thread-safe	Notes
`Collections.synchronizedMap/List/Set(x)`	One lock around every method	Iteration still needs a manual `synchronized(x)` block; compound actions aren't atomic
`Hashtable`, `Vector`	Synchronized methods	Legacy; same limitations
`ConcurrentHashMap`	CAS + locking per bucket (bin)	High concurrency; atomic `merge`/`compute`; weakly consistent iterators
`CopyOnWriteArrayList` / `CopyOnWriteArraySet`	Copies the whole array on each write	Very fast reads, slow writes; snapshot iterators
`ConcurrentLinkedQueue` / `ConcurrentLinkedDeque`	Lock-free (CAS)	Non-blocking queues
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`, `PriorityBlockingQueue`, `DelayQueue`)	Locks + conditions	Producer–consumer (Phase 11)
`ConcurrentSkipListMap` / `Set`	Lock-free skip list	Concurrent **sorted** collections

ConcurrentHashMap in depth

- **Java 7**: the table was split into 16 **segments**, each with its own lock (so up to 16 concurrent writers).
- **Java 8+**: segments are gone. An empty bin is filled with **CAS** (no lock); otherwise the thread does `synchronized` on that **bin's first node** only. Reads are **lock-free** (volatile reads). Resizing is done

cooperatively by several threads. Tree bins exist here too.

- **No null keys or values**: `get(k)` returning `null` must unambiguously mean "absent". In a concurrent map you can't safely follow it with a `containsKey` check, because another thread may change the map in between.
- Iterators are **weakly consistent**: no CME, and they may or may not show concurrent updates.
- `size()` is an estimate under concurrent updates; `mappingCount()` returns a `long`.

```
ConcurrentHashMap<String, Long> hits = new ConcurrentHashMap<>();

// WRONG under concurrency: check-then-act race between get and put
Long cur = hits.get("/login");
hits.put("/login", cur == null ? 1 : cur + 1);

// RIGHT: atomic per-key operations
hits.merge("/login", 1L, Long::sum);
hits.compute("/pay", (k, v) -> v == null ? 1 : v + 1);
hits.computeIfAbsent("/home", k -> 0L);
hits.putIfAbsent("/logout", 0L);
```

PITFALL

Even with a thread-safe map, `if (!map.containsKey(k)) map.put(k, v);` is a **race** (two separate calls). Use the atomic methods `putIfAbsent`, `computeIfAbsent` and `merge`. The same applies to `Collections.synchronizedMap`.

Fail-fast vs fail-safe (recap)

	Fail-fast	Fail-safe (snapshot / weakly consistent)
Examples	`HashMap`, `ArrayList`, `HashSet`, `TreeMap`	`ConcurrentHashMap`, `CopyOnWriteArrayList`, `ConcurrentSkipListMap`
Modify during iteration	`ConcurrentModificationException`	Allowed, no exception
Mechanism	`modCount` check	Snapshot copy (COW) or concurrent design

14.9 Sequenced Collections (Java 21)

New interfaces give a **uniform API for the first and last elements** of ordered collections.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.getFirst(); list.getLast();         // 1, 3   (instead of get(0), get(size()-1))
list.addFirst(0); list.removeLast();     // [0, 1, 2]
list.reversed();                         // [2, 1, 0]: a reversed VIEW

LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("A", 1); lhm.put("B", 2); lhm.put("C", 3);
lhm.firstEntry(); lhm.lastEntry();       // A=1, C=3
lhm.pollFirstEntry();                    // removes A=1
lhm.putFirst("Z", 26);                   // {Z=26, B=2, C=3}
lhm.sequencedKeySet().reversed();        // [C, B, Z]
```

Interface	Implemented by
`SequencedCollection`	`List`, `Deque`, `LinkedHashSet`, `SortedSet`
`SequencedSet`	`LinkedHashSet`, `SortedSet`/`TreeSet`
`SequencedMap`	`LinkedHashMap`, `SortedMap`/`TreeMap`

14.10 Choosing the Right Collection

Need	Use
Ordered list, fast index access (the default list)	`ArrayList`
Queue or stack	`ArrayDeque`
Process by priority (top-k, scheduling)	`PriorityQueue`
Unique elements, fast lookup	`HashSet`
Unique + insertion order (dedupe while keeping order)	`LinkedHashSet`
Unique + sorted + range queries	`TreeSet`
Key-value lookup (the default map)	`HashMap`
Key-value + insertion order / LRU cache	`LinkedHashMap`
Key-value + sorted keys / slabs / ranges	`TreeMap`
Enum keys or elements	`EnumMap` / `EnumSet`
Multi-threaded map, counters, caches	`ConcurrentHashMap`
Read-mostly list shared across threads	`CopyOnWriteArrayList`
Producer–consumer hand-off	`ArrayBlockingQueue` / `LinkedBlockingQueue`
Fixed constants that must not change	`List.of` / `Set.of` / `Map.of`

14.11 Big-O Cheat Sheet

Collection	add / put	get / lookup	contains	remove	Ordered?
ArrayList	O(1)* end, O(n) middle	O(1) by index	O(n)	O(n)	Insertion
LinkedList	O(1) ends	O(n)	O(n)	O(1) at ends / iterator	Insertion
ArrayDeque	O(1)* ends	O(1) ends	O(n)	O(1) ends	Insertion
PriorityQueue	O(log n)	O(1) peek	O(n)	O(log n) poll	Heap order
HashSet / HashMap	O(1)	O(1)	O(1)	O(1)	No
LinkedHashSet / LinkedHashMap	O(1)	O(1)	O(1)	O(1)	Insertion / access
TreeSet / TreeMap	O(log n)	O(log n)	O(log n)	O(log n)	Sorted
ConcurrentHashMap	O(1)	O(1)	O(1)	O(1)	No

(* = amortised. Hash-based worst case is O(log n) with tree bins.)

14.12 Classic Map Programs

```
// 1. Character frequency, sorted by character
Map<Character, Integer> freq = new TreeMap<>();
for (char c : "banking".toCharArray()) freq.merge(c, 1, Integer::sum);
System.out.println(freq);                       // {a=1, b=1, g=1, i=1, k=1, n=2}

// 2. First non-repeating character (LinkedHashMap keeps the order)
Map<Character, Integer> lf = new LinkedHashMap<>();
for (char c : "swiss".toCharArray()) lf.merge(c, 1, Integer::sum);
lf.entrySet().stream().filter(e -> e.getValue() == 1)
  .findFirst().ifPresent(e -> System.out.println(e.getKey()));   // w

// 3. Two-sum in O(n): indices of two numbers adding up to the target
int[] nums = {2, 7, 11, 15}; int target = 9;
Map<Integer, Integer> seen = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    Integer j = seen.get(target - nums[i]);
    if (j != null) { System.out.println(j + "," + i); break; }   // 0,1
    seen.put(nums[i], i);
}

// 4. Group anagrams
Map<String, List<String>> groups = new HashMap<>();
for (String w : List.of("eat", "tea", "tan", "ate", "nat")) {
    char[] k = w.toCharArray(); Arrays.sort(k);
    groups.computeIfAbsent(new String(k), x -> new ArrayList<>()).add(w);
}
// {aet=[eat, tea, ate], ant=[tan, nat]}

// 5. Sort a map by value (descending)
Map<String, Integer> sales = Map.of("Pune", 40, "Delhi", 75, "Goa", 20);
sales.entrySet().stream()
     .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
     .forEach(e -> System.out.print(e.getKey() + " "));       // Delhi Pune Goa
```

14.13 Predict the Output

```
Map<String, Integer> m = new HashMap<>();
System.out.println(m.put("a", 1) + " " + m.put("a", 2) + " " + m.get("a"));  // null 1 2

Map<String, Integer> tm = new TreeMap<>(Map.of("b", 2, "a", 1, "c", 3));
System.out.println(tm.keySet());                    // [a, b, c]

Map<Integer, String> h = new HashMap<>();
h.put(null, "x"); h.put(1, null);
System.out.println(h.get(null) + " " + h.containsKey(1) + " " + h.get(1));  // x true null

try { new ConcurrentHashMap<String, String>().put("k", null); }
catch (NullPointerException e) { System.out.println("CHM null NPE"); }      // CHM null NPE

try { new Hashtable<String, String>().put(null, "v"); }
catch (NullPointerException e) { System.out.println("Hashtable null NPE"); }

Map<String, Integer> counts = new HashMap<>();
counts.merge("x", 5, Integer::sum);
counts.merge("x", 5, Integer::sum);
System.out.println(counts);                         // {x=10}

try {
    Map<String, Integer> map = new HashMap<>(Map.of("a", 1, "b", 2));
    for (String k : map.keySet()) map.remove(k);
} catch (ConcurrentModificationException e) { System.out.println("CME"); }  // CME
```

14.14 Interview Questions

Q. How does HashMap work internally?

It keeps an array of buckets. `put` computes `hash = h ^ (h >>> 16)` from `hashCode()`, picks the bucket `(n-1) & hash`, and either inserts a node or walks the bucket's chain comparing hash and `equals()` to replace an existing key's value. Chains become red-black trees at 8 nodes (when capacity ≥ 64). When size exceeds capacity × 0.75, it doubles and redistributes.

Q. What is a collision, and how does HashMap handle it?

Different keys landing in the same bucket. HashMap chains them in a linked list, and converts long chains to balanced trees (Java 8+), so the worst-case lookup is O(log n) instead of O(n).

Q. What are the load factor and the initial capacity? What happens on resize?

The defaults are 16 buckets and a load factor of 0.75 (threshold 12). Exceeding the threshold doubles the table; each node either stays at its index or moves to index + oldCapacity.

Q. Why must keys override `equals()` and `hashCode()`? Why should keys be immutable?

Lookups find the bucket with `hashCode()` and match keys with `equals()`. If a key's fields change after insertion, its hash no longer matches its bucket, so the entry can't be found.

Q. Can HashMap have a null key? What about ConcurrentHashMap and Hashtable?

HashMap allows one null key and many null values. ConcurrentHashMap and Hashtable allow neither. For ConcurrentHashMap this prevents ambiguity of `get` returning null under concurrency.

Q. What is the difference between HashMap, LinkedHashMap and TreeMap?

No order with O(1) operations; insertion (or access) order with O(1) operations; sorted keys with O(log n) operations and navigation methods.

Q. How would you implement an LRU cache?

Extend `LinkedHashMap` with `accessOrder = true` and override `removeEldestEntry` to return `size() > capacity`. (Without that class: a HashMap plus a doubly linked list.)

Q. What is the difference between HashMap and Hashtable?

Hashtable is legacy and synchronized on every method, and allows no nulls. HashMap is unsynchronized, faster, allows one null key, and has fail-fast iterators. Use ConcurrentHashMap for thread safety.

Q. How is ConcurrentHashMap thread-safe? How did it change in Java 8?

Java 7 used segment locks. Java 8 uses CAS for empty bins and synchronizes only on the head node of an affected bin; reads are lock-free and resizing is cooperative. Compound operations like `merge`/`compute` are atomic per key.

Q. What is the difference between `Collections.synchronizedMap` and ConcurrentHashMap?

`synchronizedMap` wraps every call in one lock, so throughput is low and iteration requires manual locking. ConcurrentHashMap allows concurrent reads and writes to different bins, has weakly consistent iterators, and offers atomic compound methods.

Q. What are IdentityHashMap and WeakHashMap used for?

IdentityHashMap compares keys by reference (`==`), which suits identity-sensitive algorithms. WeakHashMap holds weak keys, so entries vanish once a key is otherwise unreachable, which suits metadata caches.

Q. What is the difference between `putIfAbsent` and `computeIfAbsent`?

`putIfAbsent(k, v)` takes an already-created value (it is evaluated even if unused). `computeIfAbsent(k, fn)` creates the value lazily, only when the key is missing, which is ideal for expensive objects and multimaps.

14.15 Quick Revision

- Map = unique keys → values; not a Collection. `put` replaces and returns the old value.
- Java 8 methods: `getOrDefault`, `putIfAbsent`, `computeIfAbsent` (multimap), `merge` (counters), `compute`, `replaceAll`.
- Iterate with `entrySet()`; remove safely with `entrySet().removeIf(...)`.
- HashMap: Node[] buckets; `hash ^ (hash >>> 16)`; index `(n-1) & hash`; collisions chain → tree at 8 (capacity ≥ 64); load factor 0.75; resize ×2; one null key.
- Keys: correct `equals` + `hashCode`, immutable (String, records, enums).
- LinkedHashMap: insertion or access order; LRU = access order + `removeEldestEntry`.
- TreeMap: sorted keys, O(log n), `floorEntry`/`ceilingKey`/`headMap`/`tailMap`/`subMap`; no null keys.
- Hashtable: legacy, synchronized, no nulls → use ConcurrentHashMap.
- IdentityHashMap uses `==`; WeakHashMap entries disappear after the key is GC'd.
- ConcurrentHashMap (Java 8): CAS + per-bin locks, lock-free reads, no nulls, atomic `merge`/`compute`, weakly consistent iterators.
- Check-then-act on any map is a race → use `putIfAbsent`, `computeIfAbsent`, `merge`.
- Java 21 sequenced collections: `getFirst`/`getLast`/`reversed`, `firstEntry`/`putFirst` on LinkedHashMap.
- Defaults: `ArrayList`, `HashMap`, `HashSet`, `ArrayDeque`; sorted → Tree*; ordered → Linked*; concurrent → `ConcurrentHashMap`.